

Nomenclature Inconsistency Review

Bridge Docs Cross-checked Against VSIM_REFERENCE.md v5.0.2

Doc ID	REV-2026-01
Scope	vsim_bridge_doc.tex, vsim_code_request.tex, vsim_bridge_audit.tex
Reference	VSIM_REFERENCE.md (v5.0.2)
Date	2026-06-01
Status	Open — findings require author action

Severity guide: **CRITICAL** = conflicts directly with reference; breaks existing parser or scripts

SIGNIFICANT = struct/TOML mismatch or naming drift; will cause silent bugs

MINOR = style inconsistency or ambiguity; no runtime impact

Contents

§1 Critical Findings	2
§2 Significant Findings	3
§3 Minor Findings	5
§4 Summary Table	6

§1 Critical Findings

F1 `[observe].track` — wrong key name

CRITICAL

Location: `vsim_bridge_doc.tex` §9 demo script (line 380); `vsim_code_request.tex` CR-08 task description.

Used: `track = ["contact", "residence", ...]`

Reference says: `metrics = ["energy_map", "rdf", ...]`

VSIM_REFERENCE.md § `[observe]` is unambiguous — the key is `metrics`, not `track`. Every existing suite script (`suite_gas_plasma`, `suite_metal_steam`) uses `metrics`. A parser will either reject `track` or silently ignore it, meaning the `identity_entropy` observable would never activate. CR-08 propagates the same wrong key in its task description.

Fix: Replace all occurrences of `track` with `metrics` in the demo script and in CR-08.

F2 `[system]` block conflicts with chemistry legacy alias

CRITICAL

Location: `vsim_bridge_doc.tex` §9 demo script (lines 357–359).

Used:

```
[system] mode = "surface_interaction" space = "3d"
```

Reference says: `[system]` is the *legacy alias* for `[chemistry]` / `[chemistry].chemistry`. It is already a parsed section with completely different semantics (ambient reaction rules, heat gate, scoring thresholds). Using `[system]` with `mode` and `space` keys will be parsed as chemistry configuration and produce undefined behaviour.

Fix: Rename to a non-conflicting block name. Candidates: `[run.surface]`, `[surface_mode]`, or promote these fields into `[run]` (which already has `mode = "md"`). The `[[surface]]` array-of-tables block below it should remain.

F3 Non-standard metric names in `[observe].metrics`

CRITICAL

Location: `vsim_bridge_doc.tex` §9 demo script.

Used: `"contact", "residence", "embedding", "diffusion_axis"`

None of these appear in the VSIM_REFERENCE.md observe metrics table. The closest match is `"diffusion_proxy"` (reference) vs `"diffusion_axis"` (bridge doc) — different names for what may be the same concept. `"contact"`, `"residence"`, and `"embedding"` appear to be surface-interaction-specific observables not yet registered in the reference.

Fix: Either register these names in the observe metrics table in VSIM_REFERENCE.md (if they are real planned metrics), or remove them from the demo script. Replace `"diffusion_axis"` with `"diffusion_proxy"` if the intent is the existing metric.

F4 `[particles.projectile]` is undocumented; conflicts with `[[simulation.molecule]]` **CRITICAL**

Location: `vsim_bridge_doc.tex` §9 demo script (lines 366–370).

Used:

```
[particles.projectile]
count = 128 type = "generic_bead" velocity = [0.0, 0.0, -1.5]
height = 15.0
```

The canonical way to declare particle species in VSIM is `[[simulation.molecule]]` (with `formula`, `count`, `temperature`, `lattice`). The bridge doc introduces `[particles.projectile]` with entirely different field names (`type` instead of `formula`; `height` and `velocity` as top-level keys instead of via `[[override.particle]]`). The parser will not recognise this block.

Fix: Replace with `[[simulation.molecule]]` + initial velocity via `[[override.particle]]` (which IS in the reference), or register `[particles.*]` as a new section type in the reference and parser.

§2 Significant Findings

F5 `surface_state` field in TOML missing from `FormationContext` C++ struct **SIGNIFICANT**

Location: §9 demo script vs §10 C++ header.

Demo script `[formation.initial_state]` contains: `surface_state = "roughened"`

The `FormationContext` struct in §10 has fields: `method`, `history`, `formation_time`, `temperature_K`, `pressure_Pa`, `phase`, `defect_state`, `grain_state`, `density` — **no** `surface_state`.

CR-01 in the code request also lists the same nine fields with no `surface_state`. The field would be silently dropped by the parser. Either add it to the struct and the CR-01 field list, or remove it from the TOML example.

F6 TOML field name `temperature` vs C++ struct field `temperature_K` **SIGNIFICANT**

Location: §4 minimum formation block vs §10 C++ struct.

§4 TOML block: `temperature = 300.0` (no unit suffix)

§10 C++ struct: `double temperature_K = 300.0;` (with `_K` suffix)

The §9 demo script correctly uses `temperature = 300.0` in `[formation.initial_state]`, which maps to the C++ field `temperature_K`. This TOML-to-struct name mismatch must be documented in the parser (similar to how `[run]` accepts both `temperature` and `temperature_K`). The §4 minimum block should note the unit convention or use the suffixed name for clarity.

F7 TOML field `pressure` vs C++ struct field `pressure_Pa`**SIGNIFICANT****Location:** §4 minimum formation block vs §10 C++ struct.

Same issue as F6: the TOML uses `pressure = 101325.0` while the struct declares `double pressure_Pa = 101325.0;`. Note also that [run] uses `pressure_GPa` (different unit), creating three pressure representations across the system. The formation block implicitly uses Pascals but the field name does not say so. The unit should be either embedded in the TOML key (`pressure_Pa`) or documented in a comment.

F8 Column header abbreviations in `.ie` data row do not match field names**SIGNIFICANT****Location:** `vsim_bridge_doc.tex` §6 data row example.

The example header uses shortened names that do not match the formal field names written by `IEWriter::write_header()` in §10:

Data row (abbreviated)	Formal field name
<code>hidden_act</code>	<code>hidden_activity</code>
<code>proj_loss</code>	<code>projection_loss</code>
<code>recov</code>	<code>recoverability</code>
<code>coal_loss</code>	<code>coalescence_loss</code>
<code>lin</code>	<code>lineage_depth</code>
<code>state_q</code>	<code>state_quota</code>
<code>entr_flux</code>	<code>entropy_flux</code>
<code>form_mem</code>	<code>formation_memory</code>

The actual `.ie` file written by `IEWriter` uses full names (confirmed in §10 `write_header()`). The example would mislead anyone using it to write a parser for the `.ie` format.

Fix: Use full field names in the data row example, or note explicitly that the header row is abbreviated for display purposes only.

F9 `[seed].formation` vs new `[formation]` block — namespace collision risk**SIGNIFICANT****Location:** `VSIM_REFERENCE.md` § `[seed]` vs bridge doc §4.

The existing `[seed]` section has a field `formation = 7306` (a 64-bit RNG seed component). The bridge doc introduces a new top-level block `[formation]` with entirely different semantics (material history context). The name proximity is a parser confusion risk: a naive reader of `seed.formation = 7306` may conflate this with the new `[formation]` block. The suite scripts already use both: `[seed] formation = 8106` alongside the intended new `[formation]` block. The reference should add a disambiguation note, and the bridge doc should acknowledge this naming overlap.

F10 `[[surface]]` block is undocumented in `VSIM_REFERENCE.md`**SIGNIFICANT****Location:** `vsim_bridge_doc.tex` §9 demo script.

The demo uses a `[[surface]]` block (array-of-tables) with fields `type = "graphite_layer"`,

```
roughness = 0.05, warm_minimum_beads = 100.
```

`[[surface]]` (array-of-tables), its fields `roughness`, `warm_minimum_beads`, and the type value `"graphite_layer"` do not appear anywhere in `VSIM_REFERENCE.md`. Either these are planned future additions that need registering, or the block name conflicts with something not yet surfaced in the reference.

§3 Minor Findings

F11 `defect_state` (field) vs “defect seed” (prose) — semantic ambiguity

MINOR

Location: §8 formation descriptors list vs §10 C++ struct.

The formation descriptors table in §8 lists “defect seed” as a descriptor type. The `FormationContext` struct field is `defect_state`. These are different concepts: a *seed* is a numerical RNG input for defect placement; a *state* is the current defect configuration. The `VSIM_REFERENCE` `[seed]` section already has a `defect seed` field (`defect = 7303`). Recommend changing “defect seed” in the §8 prose to “defect state” to match the struct, or renaming the struct field to `defect_seed` if the intent is to store a seed value.

F12 `formation_time` units unspecified

MINOR

Location: §4 minimum formation block.

`formation_time = 0.0` has no documented unit. All time-related fields in the reference use explicit unit suffixes (e.g., `dt_fs` in femtoseconds, `step_delay_ms` in milliseconds). Recommend either renaming to `formation_time_fs` or adding a comment in the TOML schema.

F13 `[outputs.identity_entropy]` not following `[export.*]` convention

MINOR

Location: All bridge docs.

Already flagged as DEC-01 in the code request, but worth stating explicitly as a nomenclature finding. All existing output configuration uses the `[export]` and `[export.visual]` namespace. The bridge doc introduces `[outputs.*]`, a new top-level namespace that does not exist elsewhere in the reference. This creates two parallel output configuration systems with no clear rule for which one owns what. DEC-01 must be resolved and the decision recorded in the reference.

F14 `vsim_bridge_audit.tex` audit tables use “Wired” where reference uses ✓

MINOR

Location: `vsim_bridge_audit.tex` §3 Runtime Wire Audit.

The audit doc uses the pill **WIRED** for existing functions. The `VSIM_REFERENCE.md` uses the symbol legend: ✓ (parsed and wired), * (parsed, not wired), × (not parsed). The audit doc should note that its status pills do not correspond to the reference symbols, or adopt the same convention to reduce cross-document confusion.

F15 [run].temperature_K vs [formation.initial_state].temperature**MINOR****Location:** §9 demo script.

The §9 demo uses `temperature_K = 300.0` in `[run]` and `temperature = 300.0` in `[formation.initial_state]`. These are different field names for the same physical quantity (temperature in Kelvin). The reference notes that `[run]` accepts both `temperature` and `temperature_K` as aliases, so both are technically valid, but mixing them within a single script increases read-time confusion. Recommend using the suffixed form (`temperature_K`) consistently across all blocks in new scripts.

F16 C++ IERRecord.state_quota is uint32_t but lineage_depth range may exceed 32-bit on deep coalescence chains**MINOR****Location:** §10 C++ header.

`uint32_t lineage_depth = 0;` and `uint32_t state_quota = 1;` are both 32-bit unsigned. For `state_quota` (Q_n , particle count), `uint32_t` caps at $\approx 4 \times 10^9$, which is sufficient. For `lineage_depth`, the cap is also fine in practice. However, the `frame` and `identity_id` fields are `uint64_t`. The mixed 32/64-bit usage should be documented as a deliberate choice in the header, or both counters widened to `uint64_t` for consistency.

§4 Summary Table

F#	Severity	Title	Location
F1	CRITICAL	[observe].track wrong key name	bridge §9, CR-08
F2	CRITICAL	[system] conflicts with chemistry alias	bridge §9
F3	CRITICAL	Non-standard observe metric names	bridge §9
F4	CRITICAL	[particles.projectile] undocumented	bridge §9
F5	SIGNIFICANT	surface_state in TOML, absent from struct	bridge §9, §10
F6	SIGNIFICANT	TOML temperature vs struct temperature_K	bridge §4, §10
F7	SIGNIFICANT	TOML pressure vs struct pressure_Pa	bridge §4, §10
F8	SIGNIFICANT	Column header abbreviations don't match field names	bridge §6
F9	SIGNIFICANT	[seed].formation vs [formation] collision risk	ref + bridge §4
F10	SIGNIFICANT	[[surface]] block undocumented	bridge §9
F11	MINOR	defect_state vs "defect seed" prose	bridge §8, §10
F12	MINOR	formation_time units unspecified	bridge §4
F13	MINOR	[outputs.*] vs [export.*] namespace	all bridge docs
F14	MINOR	Audit status pills vs reference symbol legend	audit §3
F15	MINOR	Mixed temperature / temperature_K in one script	bridge §9

F#	Severity	Title	Location
F16	MINOR	Mixed <code>uint32_t</code> / <code>uint64_t</code> widths in IERecord	bridge § 10

Recommended action order: Fix F1 (wrong observe key) and F2 (system block conflict) immediately — both would cause a real script to silently misbehave or error at parse time. F3 and F4 (undocumented blocks/metrics) should be resolved before the bridge doc § 9 script is used as a canonical example. F5–F10 should be resolved before CR-01 is closed.